CS 498

1. **Row Key & Sharding:** In Bigtable, data is dynamically partitioned (sharded) across nodes based on the Row Key. What is your proposed Row Key for this dataset? Explain how this choice avoids "hotspotting" and impacts your ability to efficiently query vehicles by *City* or *Make*.
    1. The proposed Row Key for this dataset is City#Make#VIN. I chose this row key because Bigtable stores rows in sorted order by row key, so putting City first allows me to easily query all vehicles in a specific city using a prefix scan. Then, by adding Make after, I can narrow it down further. Finally, I added VIN at the end to make sure each row is unique, since VIN is unique for each vehicle. This design allows us to efficiently query vehicles by city or by city and make. However, this design is not efficient for querying by make alone. In addition, it could still cause hotspotting. For example, if a city has a lot of vehicles, then many rows will share the same prefix and end up in the same range. This can lead to uneven load across nodes.
2. **Column Families:** How would you design the Column Families for this dataset? Group the attributes logically and explain how this aids in distributed read performance.
    1. For this dataset, I would create id, location, vehicle, and program column families. The id column family would have vehicle identifiers such as VIN and Vehicle ID. The location column family would have location-related information such as City, Postal Code, and Vehicle Location. The vehicle column family would have vehicle-related information such as Make, Model, and Electric Vehicle Type. Finally, the program column family would have information about any special programs or policies, such as CAFV Eligibility and Electric Utility. I grouped the column families this way because the information in each group is often accessed together in queries. This improves read performance because Bigtable stores each column family separately on disk, so a query can read only the relevant column families instead of the entire row. This reduces unnecessary data reads and lowers I/O. As a result, each node in the distributed system only processes the data it needs, which makes reads more efficient overall.
3. **Horizontal Scaling:** Compare the pros and cons of using a *Hashed Shard Key* (e.g., on VIN) versus a *Range-Based Shard Key* (e.g., on Model Year) regarding query routing efficiency and cluster performance.

	Pros	Cons

Hashed Shard Key	A hashed shard key spreads the data evenly across the cluster, meaning that the data gets distributed across many nodes instead of just one. This helps prevent hotspotting because no single node is overloaded with too many reads or writes. Load balancing is also improved.	Query routing is less efficient because related rows are spread across many shards and the system can't easily perform a single range scan for queries. Instead, the query often needs to contact multiple nodes and combine the results. This increases the amount of work needed and can lead to higher latency.
Range-Based Shard Key	A range-based shard key groups similar values together, which makes query routing more efficient when queries use that same field. The system can also perform a single range scan easier compared to the hashed shard key. This makes reads more direct and faster for queries that use the same field as the shard key, such as filtering by a specific value or range of that field.	A range-based shard key can lead to uneven load across the cluster. If certain values are much more common than others, such as newer model years, then a large portion of the data will be stored in a small number of key ranges. This can cause hotspotting, where some nodes receive a much higher amount of reads and writes than others. So, the system might not be able to scale evenly which can lead to a decrease in performance.